05 — control_unit.v Explained: The Instruction Decoder

Oct 6, 2026 · @shiv

1. Overview

`control_unit.v` reads the 4-bit opcode and switches on the right control signals, like a switchboard. It is pure combinational logic (a hard-wired control unit). Its signals are made in Stage 2 (ID) and then carried down the pipeline registers to the stages that use them.

Item	Value
File	design_sources/control_unit.v (101 lines)
Module name	control_unit
Pipeline stage	Stage 2 – Instruction Decode (ID)
Type of logic	Combinational (a lookup table)
Calls (instantiates)	Nothing – leaf module
Called by	risc8_5stage_core.v, instance u_ctrl, lines 96–112
Its outputs go to	alu.v (alu_op, via ID/EX), hazard_unit.v (uses_a, uses_b), and the pipeline registers

1.1 Where it sits

```
tb_risc8_5stage.v
└── risc8_5stage_core.v
    ├── imem.v
    ├── control_unit.v   <== THIS FILE
    ├── regfile.v
    ├── hazard_unit.v
    ├── alu.v
    └── dmem.v
```

1.2 Ports

Port	Dir.	Width	Meaning when 1	Goes to (in risc8_5stage_core.v)	Used in stage
opcode	in	4	Instruction bits [15:12]	from ifid_instr[15:12]	ID
alu_op	out	3	Which ALU operation (0–7)	id_alu_op → idex_alu_op → alu.v	EX
alu_src_imm	out	1	ALU input B is the immediate	idex_alu_src_imm	EX
ra_is_rd	out	1	Read port A reads rd (ADDI)	selects id_ra_addr	ID
rb_is_rd	out	1	Read port B reads rd (ST data)	selects id_rb_addr	ID
uses_a	out	1	Instruction really reads port A	hazard_unit.v (id_uses_a, ex_uses_a)	ID, EX
uses_b	out	1	Instruction really reads port B	hazard_unit.v (id_uses_b, ex_uses_b)	ID, EX
reg_we	out	1	Write a register in WB	idex → exmem → memwb_reg_we	WB
mem_to_reg	out	1	Result comes from memory (LD)	idex → exmem_mem_to_reg	MEM
mem_we	out	1	Write data memory (ST)	idex → exmem_mem_we	MEM
flag_we	out	1	Update Z and C	idex_flag_we	EX
jump	out	1	Instruction is JMP	idex_jump	EX
br_z	out	1	Instruction is BZ	idex_br_z	EX
br_nz	out	1	Instruction is BNZ	idex_br_nz	EX
halt	out	1	Instruction is HLT	idex_halt	EX → WB

1.3 Dependencies

Modules, functions or tasks called inside this file: none. It is a leaf module.

Used by (file)	How	Instance	Lines
risc8_5stage_core.v	Module instance	u_ctrl	96–112

Must agree with: alu.v (the alu_op numbers 0–7) and the opcode table in imem.v (opcodes 0–F).

1.4 Verilog constructs used

Construct	Meaning in plain words
`output reg`	Output assigned inside an always block (still combinational here)
`localparam`	Named constants for the 16 opcodes
`always @(*)`	Combinational block
Default assignments	Every output gets a value first, so no latches are created
`case` / `endcase`	Choose actions by opcode
`begin ... end`	Groups several statements into one

2. Code walkthrough, part by part

2.1 File header (lines 1–31)

The header explains that the decoder turns the opcode into control signals, and which stage uses each group. It lists the files connected to it: risc8_5stage_core.v creates it, alu.v uses alu_op, and hazard_unit.v uses uses_a and uses_b. It also contains the opcode map, which must match the program in imem.v.

2.2 Module and ports (lines 33–51)

```
`timescale 1ns/1ps

module control_unit (
    input  wire [3:0] opcode,       // instr[15:12] from IF/ID register
    output reg  [2:0] alu_op,       // ALU operation (alu.v)
    output reg        alu_src_imm,  // 1: ALU operand B = imm8
    output reg        ra_is_rd,     // 1: read port A uses rd (ADDI)
    output reg        rb_is_rd,     // 1: read port B uses rd (ST data)
    output reg        uses_a,       // 1: instruction reads port A
    output reg        uses_b,       // 1: instruction reads port B
    output reg        reg_we,       // 1: write a register in WB
    output reg        mem_to_reg,   // 1: WB data comes from memory (LD)
    output reg        mem_we,       // 1: write data memory in MEM (ST)
    output reg        flag_we,      // 1: update Z and C in EX
    output reg        jump,         // JMP
    output reg        br_z,         // BZ
    output reg        br_nz,        // BNZ
    output reg        halt          // HLT
);
```

There is one input, the 4-bit opcode, and 14 outputs. They are declared `output reg` only because they are assigned inside an always block; no flip-flops are created.

2.3 Opcode names (lines 53–55)

```
    localparam NOP=4'h0, ADD=4'h1, SUB=4'h2, AND=4'h3, OR =4'h4, XOR=4'h5,
               SHL=4'h6, SHR=4'h7, LDI=4'h8, ADDI=4'h9, LD=4'hA, ST=4'hB,
               JMP=4'hC, BZ =4'hD, BNZ=4'hE, HLT=4'hF;
```

These give a readable name to each of the 16 opcodes, so the case statement can say `ADD` instead of `4'h1`.

2.4 Step 1: defaults (lines 57–62)

```
    always @(*) begin
        // ---- STEP 1: defaults = NOP (also prevents latches) ----
        alu_op = 3'd0; alu_src_imm = 0; ra_is_rd = 0; rb_is_rd = 0;
        uses_a = 0; uses_b = 0;
        reg_we = 0; mem_to_reg = 0; mem_we = 0; flag_we = 0;
        jump = 0; br_z = 0; br_nz = 0; halt = 0;
```

`always @(*)` makes this combinational. Every output is first set to 0, which is exactly a NOP. This has two benefits:

- Each case below only has to switch **on** what it needs.
- Every output has a value on every path, so synthesis creates **no latches**.

2.5 Step 2: ALU instructions (lines 64–75)

```
        case (opcode)
            // rd = rs1 OP rs2 : reads A and B, writes rd, sets flags
            ADD : begin alu_op = 3'd0; uses_a = 1; uses_b = 1; reg_we = 1;
flag_we = 1; end
            SUB : begin alu_op = 3'd1; uses_a = 1; uses_b = 1; reg_we = 1;
flag_we = 1; end
            AND : begin alu_op = 3'd2; uses_a = 1; uses_b = 1; reg_we = 1;
flag_we = 1; end
            OR  : begin alu_op = 3'd3; uses_a = 1; uses_b = 1; reg_we = 1;
flag_we = 1; end
            XOR : begin alu_op = 3'd4; uses_a = 1; uses_b = 1; reg_we = 1;
flag_we = 1; end

            // rd = rs1 << 1 / >> 1 : reads A only
            SHL : begin alu_op = 3'd5; uses_a = 1; reg_we = 1; flag_we = 1;
end
            SHR : begin alu_op = 3'd6; uses_a = 1; reg_we = 1; flag_we = 1;
end
```

The register-register instructions read both ports, write rd and update the flags. Shifts read only port A. **Link:** the `alu_op` numbers (0–6) must match the `localparam` values in **alu.v**.

2.6 Immediate, load and store (lines 77–88)

```
            // rd = imm8 : reads no register, flags unchanged
            LDI : begin alu_op = 3'd7; alu_src_imm = 1; reg_we = 1; end

            // rd = rd + imm8 : port A reads rd
            ADDI: begin alu_op = 3'd0; alu_src_imm = 1; ra_is_rd = 1; uses_a
= 1;
                        reg_we = 1; flag_we = 1; end

            // rd = MEM[rs1] : port A gives the address
            LD  : begin uses_a = 1; reg_we = 1; mem_to_reg = 1; end

            // MEM[rs1] = rd : port A = address, port B = data (rd)
            ST  : begin uses_a = 1; uses_b = 1; rb_is_rd = 1; mem_we = 1; end
```

- **LDI** uses ALU operation 7 (pass B), with B set to the immediate (`alu_src_imm`). It reads no register.
- **ADDI** adds the immediate to rd, so port A must read rd (`ra_is_rd = 1`).
- **LD** reads the address from rs1 on port A, and its result comes from memory (`mem_to_reg`).
- **ST** reads the address on port A and the data (rd) on port B (`rb_is_rd = 1`). It writes memory, not a register.

2.7 Control flow, halt and NOP (lines 90–101)

```
            // Control flow: target = imm8, no registers read
            JMP : jump  = 1;
            BZ  : br_z  = 1;
            BNZ : br_nz = 1;

            HLT : halt  = 1;
            NOP : ;      // NOP: keep all defaults (do nothing)
            default: ;   // (all 16 opcodes are listed, so never used)
        endcase
    end

endmodule
```

JMP, BZ and BNZ only set their own flag; the core uses `imm8` as the target and decides in EX whether to jump. HLT sets `halt`. NOP keeps all the defaults. The `default` line is a safety net; it is never used because all 16 opcodes are listed.

3. Full control signal table

A blank cell means 0. A dash for alu_op means the ALU result is not used.

Instr.	alu_op	alu_src_imm	ra_is_rd	rb_is_rd	uses_a	uses_b	reg_we	mem_to_reg	mem_we	flag_we	Branch / halt
NOP	–										
ADD	0				1	1	1			1	
SUB	1				1	1	1			1	
AND	2				1	1	1			1	
OR	3				1	1	1			1	
XOR	4				1	1	1			1	
SHL	5				1		1			1	
SHR	6				1		1			1	
LDI	7	1					1				
ADDI	0	1	1		1		1			1	
LD	–				1		1	1			
ST	–			1	1	1			1		
JMP	–										jump
BZ	–										br_z
BNZ	–										br_nz
HLT	–										halt

4. Common mistakes

- Forgetting the default assignments: synthesis creates latches, and undefined opcodes behave randomly.
- Changing an alu_op number here without changing alu.v.
- Setting uses_a or uses_b for an instruction that does not read that register: the hazard unit then causes needless stalls.
- Setting reg_we for ST or for branches: they would overwrite a register by mistake.

5. Viva questions

Q1. Is this a hard-wired or a microprogrammed control unit? Hard-wired: it is combinational logic built from a case statement, which is typical for RISC.

Q2. Why are all outputs set to 0 first? So that every output has a value on every path, which avoids latches. It also means each case only switches on what it needs.

Q3. Why does ADDI set ra_is_rd? ADDI computes rd = rd + imm8, so read port A must read rd instead of rs1.

Q4. What are uses_a and uses_b for? They tell the hazard unit which registers an instruction really reads, so forwarding and stalls happen only when needed.

Q5. In which stage are the control signals used? They are made in ID and carried along: ALU and branch signals are used in EX, memory signals in MEM, and reg_we in WB.